

Connecting Needs: Global Specification

A Global Protocol for INUO (iNeed-Uoffer, or the PLATFORM) Matching

1. System Architecture Overview

The PLATFORM is an interaction protocol implemented via a distributed event-driven architecture. It provides a standardized engine for connecting localized social requirements with fulfillment entities by abstracting human intent into canonical data structures.

2. Core Interaction Engine

The engine utilizes a structured formulation of human intent where every interaction is modeled as a `Need` object, defined by the formula: **NEED = (VERB) + (OBJECT)**.

- **Canonical Catalog:** All interaction objects (Product, Service, or Social Interaction) must map to a centralized Global Catalog to ensure canonical data consistency and prevent namespace collision.
- **Verb/Complement Logic:** A **MATCH** is programmatically validated by the systematic connection between a `VERB` (Request) and its corresponding `COMP_VERB` (Offer). The engine enforces these pairings to maintain state integrity.
- **Model Isolation:** The system maintains strict architectural boundaries between Transactional (Commercial) and Gift-Based (Altruistic) models to prevent cross-contamination of fulfillment logic.

3. Macro-Need Decomposition & Dependency Graphs

Complex objectives (Macro-Needs) are decomposed into **Atomic Needs**, representing the fundamental unit of actionable transaction. These are organized into a **Dependency Graph** where a Parent Need remains in a `Blocked` state until its Prerequisite Needs are resolved.

4. Platform Governance & Safety

The platform enforces a Trust Loop and Zero-Tolerance policy regarding illegal activities. Prohibited content includes human trafficking and exploitation.

- **Identity Verification:** Mandatory for users engaging in sensitive domains (e.g., medical support, security) to ensure ecosystem accountability.
- **Immutable Audit Trail:** Interaction logs and communications are preserved to enable dispute resolution and moderator review; deletion of sent messages is restricted.
- **Enforcement:** Automated policy enforcement triggers temporary or permanent suspension of entities failing to adhere to legal frameworks or safety protocols.

5. Infrastructure & Connectivity

The platform implements a network-aware API architecture for operational continuity in diverse environments, utilizing an offline-first and asynchronous approach.

- **Low-Bandwidth Protocols:** Support for SMS/USSD gateways and minimal binary payload adaptation to optimize throughput on restricted connections.
- **Asynchronous API Gateway:** Utilizes a "Store-and-Forward" protocol and absolute timestamp reconciliation to manage intermittent synchronization and variable network latencies.

6. Ecosystem Integration

The PLATFORM leverages an **Adapter Pattern** to integrate external service ecosystems (e.g., Uber, LinkedIn) as fulfillment providers.

- **LLM-Broker Middleware:** Serves as integration intelligence, mapping structured Needs to external API payloads (JSON/REST) and managing OAuth 2.0 identity unification.

3.1 Macro-Need Decomposition Reference (10km Road)

- **Resilience Layer:** External APIs are treated as intermittent services; the system maintains local state and queues requests when endpoints are unreachable.

7. Recursive Development Lifecycle (The Seed Agent)

Need Verb	Complement	Example Scenario
Request	Donate	Need: Food packet Offer: Packaged meals
Buy	Sell	Need: Bicycle Offer: Used mountain bike
Seek	Offer	Need: Career mentor Offer: Industry professional
Need	Fulfill	Need: Emergency shelter Offer: Temporary housing
Borrow	Lend	Need: Construction tools Offer: Equipment loan
Consult	Advise	Need: Legal inquiry Offer: Legal counsel
Search	Supply	Need: Rare blood type Offer: Blood bank inventory
Call	Respond	Need: Crisis help Offer: Emergency support
Volunteer	Coordinate	Need: Event staff Offer: Volunteer coordination
Report	Action	Need: Road hazard Offer: Maintenance crew
Seek	Befriend	Need: New social circle Offer: Open to new friends
Meet	Host	Need: Someone to have coffee with Offer: Casual meetup
Discover	Connect	Need: Professional networking Offer: Mutual introduction

Need Verb	Complement	Example Scenario
Ride	Drive	Need: Commute to work Offer: Shared ride to city center
Talk	Listen	Need: Someone to talk to Offer: Active listener
Transport	Carry	Need: Goods relocation Offer: Trucking service
Deliver	Fetch	Need: Package delivery Offer: Local courier service
Employ	Teach	Need: School requires staff Offer: Certified teacher availability
Contract	Nurse	Need: Patient requires home visit Offer: Licensed nursing care
Recruit	Apply	Need: Organization needs help Offer: Individual seeking employment
Offer	Accept	Employer offers position Candidate accepts position
Interview	Attend	Company requests interview Candidate attends interview

10. Complex Use Case: Macro-Need Decomposition

Scenario: A community identifies a structural requirement for a new road connecting Point A and Point B. In a traditional system, this request is too broad to be matched with a single provider.

ID	Need Verb	Complement	Example Scenario
1	GOAL	N/A	Construct 10km Desert Road (incl. Tunnels/Bridges)
1.1	Consult	Advise	Need: Geotechnical survey of shifting desert sand Offer: Geological engineer
1.2.1	Request	Donate	Need: Detailed topographic surveying for structural alignment Offer: Surveying firm
1.2.2	Request	Donate	Need: Specialized bridge load calculations for sand-based pillars Offer: Structural analyst
1.2.3	Request	Donate	Need: Arid-zone tunnel ventilation and structural design Offer: Tunneling consultant
1.3	Report	Action	Need: Environmental impact assessment for desert wildlife Offer: NGO/Regulatory inspector
1.4	Borrow	Lend	Need: High-heat resistant machinery for desert excavation Offer: Heavy equipment rental
1.5	Recruit	Apply	Need: Tunneling crew for rocky sub-surface Offer: Specialized construction labor
1.6	Employ	Teach	Need: Site manager for desert logistics Offer: Project management professional

10.1 The Atomic Threshold

The **Atomic Need** represents the fundamental unit of the system, defined as the point at which a macro-need has been successfully decomposed into a singular, actionable



transaction. This state is reached when the requirement matches a specific Verb + Object/Interaction pair (e.g., 'Hire 5 engineers'). At this level, the need is ready for direct matching and marketplace fulfillment, signaling that the decomposition process is complete and the intent is fully resolvable by the interaction engine.

By fragmenting complex objectives into specific, matchable micro-tasks, the platform enables the execution of large-scale infrastructure projects that would otherwise be impossible to coordinate through simple, single-intent transactions.

10.2 Need Dependencies (The Dependency Chain)

Needs can be systematically chained to represent complex workflows. In this hierarchy, a **Parent Need** (e.g., 'Construct Road') can be set to a `Blocked` state, remaining inactive until its **Prerequisite Needs** (e.g., 'Open Tender', 'Geotechnical Survey') are successfully resolved. This architecture enables the creation of a **Dependency Graph** of atomic tasks, allowing the system to automatically 'unlock' and trigger future needs only when their specific operational requirements have been fulfilled.

11. The Self-Orchestrating Dev Lifecycle (INUO-on-INUO)

The PLATFORM leverages its own architectural logic to manage its development, maintenance, and evolution. By treating the software development lifecycle as an INUO marketplace, the PLATFORM becomes self-improving.

11.1 The Recursive Loop

Development is treated as an ongoing transaction between the Product Owner (PO) and the System:

- **PO Intent (The Need):** The PO inputs a high-level goal (e.g., "Improve system performance for low-bandwidth users").
- **Semantic Decomposition:** The LLM acts as the system architect, applying the decomposition logic (Section 10) to the requirement. It breaks the "Need" into atomic technical tasks (e.g., "Refactor compression algorithm", "Optimize database indexing").
- **Atomic Resolution (The Offer):** The LLM fulfills these atomic development "Needs" by generating code, running tests, and proposing commits (the "Offer").

11.2 The Codebase as Global Catalog

The codebase is treated as a structured catalog. Every function, service, and module is indexed by its "Verb + Object" purpose. When a "Need" arises, the LLM searches the existing codebase "catalog" to see if a match exists (reuse/refactoring) or if a new "Offer" (code implementation) is required.

11.3 Automated Quality Control

- **Feedback Loop:** Once code is deployed, the system monitors performance metrics. If metrics fall, the system creates a new "Need" (e.g., "Fix performance degradation"), effectively triggering a new development cycle without human intervention.

Constraint Enforcement: The PLATFORM's safety policies (Section 6) are applied to the code itself—preventing the injection of insecure code by treating every line of code as an interaction that must be "validated."

12. External Ecosystem Integration

The PLATFORM acts as a master-orchestrator, leveraging existing service ecosystems rather than replicating them.

12.1 The Adapter Pattern

To maintain a global, lightweight footprint, the PLATFORM utilizes an 'Adapter Pattern' architecture. Each external platform (Uber, MercadoLibre, LinkedIn, etc.) is integrated as a 'Fulfillment Provider.'

- **The PLATFORM's Role:** It maintains the logic of the 'Need' and the 'User Intent.'
- **The Adapter's Role:** It translates the PLATFORM's internal 'Match' into a specific API call (e.g., triggering an Uber ride, posting a LinkedIn job, or initiating a MercadoLibre purchase).

12.2 The LLM as the 'Universal API Broker'

The LLM serves as the integration intelligence, dynamically mapping a structured 'Need' to the correct external provider.

- **Intent-to-API Mapping:** When a match is found that requires an external service, the LLM constructs the necessary payload (JSON/REST) to trigger the target platform's API, ensuring a seamless user experience without requiring the user to leave the PLATFORM interface.

- **Identity Unification:** By leveraging OAuth 2.0 and SSO (Single Sign-On), the PLATFORM securely manages user identities across external providers, allowing the user to 'Authorize' the PLATFORM to act on their behalf within these external ecosystems.

12.3 Tech Stack & Connectivity

- **Orchestration Layer:** Event-driven architecture using a cloud-native messaging bus (e.g., Kafka or similar) to manage the state of requests being fulfilled by external APIs.
- **Gateway Layer:** A unified API Gateway that standardizes authentication, rate limiting, and error handling for all external integrations.

Resilience Protocol: The system treats external APIs as 'Intermittent Services.' It maintains a local state for every Need; if an external API (e.g., Uber) is unreachable, the PLATFORM automatically queues the request or alerts the user to manual alternatives.

The PLATFORM utilizes an **INUO-on-INUO** process where the software development lifecycle is treated as an internal marketplace.

- **Seed Agent Architecture:** A CLI-based agent parses Product Owner intent into atomic technical needs, proposes code implementations (Offers), and applies validated changes.
- **System Prompt Specification:** Defines the operational logic for the Seed Agent, mandating decomposition, verb/object categorization, and dependency tracking.
- **Self-Orchestration Loop:** Continuous monitoring of deployment metrics triggers new development Needs for performance optimization or bug fixes without human intervention.

7.4 Seed Agent Bootstrap Protocol

The initialization of the development environment requires the integration of an Agentic IDE (e.g., Cline) with the canonical specification. The bootstrap sequence is strictly defined as follows:

- **System Prompt Injection:** Load `INUO_SPEC.md` as the persistent system prompt to govern all autonomous reasoning and code generation.
- **Manifest Initialization:** Generate `inuo-manifest.json` with `SPEC_VERSION: "0.1.0"` to establish the operational baseline.
- **Structural Mapping:** Parse architecture definitions into localized directory structures, mapping "Verb + Object" pairs to discrete service modules.

- **Logic Enforcement:** Implement hard-coded constraints for Atomic Threshold validation and Dependency Chain resolution, ensuring no Parent Need is activated without Prerequisite fulfillment.

14. Versioning & Operational Lifecycle

14.1 Versioning Schema: All specifications utilize Semantic Versioning (MAJOR.MINOR.PATCH).

- **PATCH:** Document refinements and non-breaking clarifications.
- **MINOR:** Additions to the Global Catalog, new Verbs, or ecosystem integrations.
- **MAJOR:** Changes to the Core Interaction Engine (Need/Verb structure) or fundamental safety protocols.

14.2 Sync Protocol: Every CLI implementation must include a 'inuo-manifest.json' file containing the target 'SPEC_VERSION'. The Seed Agent must validate that the CLI logic matches the document's versioning before deployment.

14.3 Development Workflow:

1. **1. Define:** Update SPEC_VERSION in the document header.
2. **2. Implement:** The Seed Agent updates the CLI/Codebase to align with the new spec.
3. **3. Verify:** Execute 'inuo-cli test --version [SPEC_VERSION]'.
4. Decision: If 'Pass', tag CLI as stable. If 'Fail', execute 'inuo-cli rollback [PREVIOUS_VERSION]' to revert the codebase to the last known stable state defined by the previous document snapshot.